discussion involves the use of terms the software engineering community has assimilated to describe principles and techniques. Many of the terms are quite abstract and are used to describe abstract ideas. Consequently, they can be difficult to grasp, especially for literal thinkers.

I have already used the term *loose coupling* as one of the primary targets a software engineer should strive for when designing a system. I've mentioned that this loose coupling should occur between components. Before looking at the nature of a component, I want to take a step back and look at the bigger picture in terms of software engineering design principles.

7.1.1 Single-responsibility principle

A key principle you should consider when designing a system is the *single-responsibility principle* (SRP). This is the *S* in SOLID, a set of principles designed to make software more understandable, flexible, and maintainable. Fundamentally, SRP states that any component, module, or service within an application should have only one reason to change: its single responsibility needs to change for some business reason. If you consider a `PageModel` class with this principle in mind, you can see its responsibility is to process the HTTP request for its page. Therefore, the only circumstances under which you should need to change any aspect of the code in the `PageModel` is if the logic required for processing the request should change.

If you look at the `PageModel` for the property manager's Create page you put together in the last chapter, you can see this principle is violated. The `PageModel` has *two* responsibilities: to process the request and to generate data for the `SelectList` in the form of a collection of cities (figure 7.1).